

MÓDULO ESTRUCTURAL · LABORATORIO PRÁCTICO

Hands-On: Diseñando Sistemas Financieros Seguros con Mermaid.js

Del código al diagrama: modelado de clases y documentación técnica guiada por IA.

Herramienta: Mermaid Live Editor · mermaid.live (gratuita, sin instalación)

Requisito: navegador web únicamente

Lo mínimo que necesitas para dibujar tu primer diagrama de clases

```
classDiagram
  class Card {
    +String panMasked
    -String encryptedPAN
    +isValid() boolean
  }

  class Transaction
  Transaction "1" o-- "1" Card
```

```
%% Sintaxis de relaciones
A --> B      %% Asociación
A --|> B     %% Herencia
A ..|> B     %% Realización
A *-- B      %% Composición
A o-- B      %% Agregación
A ..> B      %% Dependencia

%% Visibilidad: + público - privado
```

- | **Miembros:** cada línea dentro de las llaves { } es un atributo o método — el signo inicial define visibilidad (+ público, - privado).
- | **Relaciones:** se escriben FUERA de las llaves, como una línea independiente entre dos nombres de clase — el símbolo del medio define el tipo de relación.
- | **Multiplicidad:** los números entre comillas ("1", "*") antes y después de la relación indican cuántas instancias participan de cada lado.

Copia este código en mermaid.live y confirma que renderiza correctamente

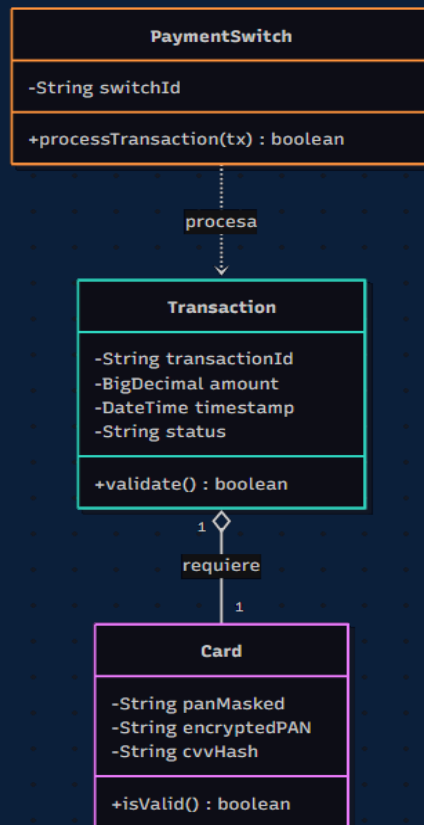
```
classDiagram
    class Card {
        -String panMasked
        -String encryptedPAN
        -String cvvHash
        +isValid() boolean
    }

    class Transaction {
        -String transactionId
        -BigDecimal amount
        -DateTime timestamp
        -String status
        +validate() boolean
    }

    class PaymentSwitch {
        -String switchId
        +processTransaction(tx) boolean
    }

    Transaction "1" o-- "1" Card : requiere
    PaymentSwitch ..> Transaction : procesa
```

clase-base.mmd · copiar en mermaid.live



Lo que deberías ver

- | 3 clases: Card, Transaction y PaymentSwitch.
- | Una agregación (o-->) entre Transaction y Card, con multiplicidad 1..1.
- | Una dependencia (..>) de PaymentSwitch hacia Transaction.
- | Ningún mecanismo de seguridad todavía — el PAN y el CVV viajan en claro. Eso es exactamente lo que vas a corregir en el siguiente paso.

Añadiendo el subsistema de anonimización

| Instrucciones: extiende tu diagrama base agregando las clases TokenizationService, CryptoHSMSERVICE y la interfaz ITokenizationEngine, con las relaciones correctas hacia Card y PaymentSwitch.

Elemento a agregar	Miembros clave	Relación requerida
«interface» ITokenizationEngine	tokenize(), detokenize(), maskPan()	TokenizationService la implementa (realización)
TokenizationService	saltKey · tokenize() · hashCVV()	Usa CryptoHSMSERVICE (dependencia); crea Token
CryptoHSMSERVICE	hsmKeySlot · encryptFPE() · decryptFPE()	Depende de ella TokenizationService
PaymentSwitch	(ya existe)	Agregar dependencia hacia ITokenizationEngine — nunca hacia la clase concreta

Pista de diseño: PaymentSwitch debe depender de la INTERFAZ (ITokenizationEngine), nunca de TokenizationService directamente. Esa es la diferencia entre un diseño que se puede auditar y uno que no — el Módulo 1 de la presentación explicó por qué.

Uso de IA por el estudiante: de tu diagrama Mermaid a un SDD completo

prompt-documentacion-ieee1016.txt

Actúa como un Ingeniero de Software Senior. A continuación, te proporciono el código Mermaid.js de mi diagrama de clases de un sistema de procesamiento de pagos con módulo de anonimización:

[PEGAR CÓDIGO MERMAID AQUÍ]

Genera un documento de explicación técnica completo bajo el estándar IEEE 1016 (Software Design Description) que contenga:

1. Resumen Ejecutivo: propósito y alcance del diseño.
2. Diccionario de Clases: cada clase, interfaz, sus atributos (justificando los tipos de datos) y sus métodos.
3. Análisis de Relaciones: justificación técnica de por qué se uso composición, agregación, herencia o dependencia en cada conexión.
4. Seguridad y Anonimización: cómo este diseño protege el PAN y el CVV bajo PCI-DSS v4.0.
5. Guía de Explicación para Evaluación: puntos clave y argumentos técnicos para la defensa del trabajo.

